UNIVERSIDADE PAULISTA - UNIP

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

GUILHERME AUGUSTO GATTO - R.A: R2798I3
LUIS HENRIQUE DE AQUINO MELLO - R.A: R2882I9
IGOR RIBEIRO MORAIS DA SILVA - R.A: R335011

PROJETO INTEGRADO MULTIDISCIPLINAR SÃO JOSÉ DOS CAMPOS 2026

SÃO JOSÉ DOS CAMPOS
2026

RESUMO

Este Projeto Integrado Multidisciplinar (PIM II) foi desenvolvido com base no planejamento de um sistema informatizado para a **Barbearia Santos**, um estabelecimento fictício criado para dar contexto prático ao nosso aprendizado acadêmico. A ideia principal do projeto é resolver problemas comuns do dia a dia de um pequeno negócio, substituindo o atendimento manual e a troca de mensagens por WhatsApp por uma ferramenta digital organizada e fácil de usar. O objetivo principal é aplicar de verdade os conhecimentos que estamos estudando nas matérias do segundo semestre do curso Superior de Análise e Desenvolvimento de Sistemas. Para tirar a ideia do papel, o projeto foi estruturado passo a passo, definindo quem pode acessar o quê no sistema — separando as funções do dono e do cliente. Com isso, o software foi planejado para permitir que os clientes façam seus próprios agendamentos e cadastros com autonomia, enquanto o proprietário consegue gerenciar os horários da barbearia, cadastrar novos serviços, acompanhar o estoque de produtos de forma simples e consultar o histórico de tudo o que já foi atendido. Dessa forma, o trabalho mostra como a tecnologia pode ser útil na rotina de uma empresa, trazendo mais organização para o fluxo de informações, segurança para os dados salvos e facilitando tanto a vida de quem administra quanto de quem consome o serviço.

Palavras-chave: Barbearia Santos, Análise e Desenvolvimento de Sistemas, Gestão de Agenda, Automação, Controle de Estoque.

RESUMO DE LINGUA ESTRANGEIRA (INGLÊS)

This Multidisciplinary Integrated Project (PIM II) was developed based on the planning of a computerized system for Barbearia Santos, a fictional business created to give practical context to our academic learning. The core idea of the project is to solve everyday challenges faced by small businesses by replacing manual service and WhatsApp text messaging with an organized, user-friendly digital tool.

The primary objective is to genuinely apply the knowledge we are studying across the courses of the second semester of the Higher Education degree in Systems Analysis and Development.

To bring the concept to life, the project was structured step-by-step, establishing clear access controls within the system—separating the functionalities for the owner and the customers. Consequently, the software was designed to allow clients to handle their own bookings and registrations autonomously, while enabling the owner to manage shop hours, register new services, easily monitor product inventory, and review the history of all completed appointments.

Ultimately, this work demonstrates how technology can benefit daily business operations, bringing greater organization to information flows, safeguarding stored data, and making life easier for both administration and customers.

Keywords: Barbearia Santos, Systems Analysis and Development, Appointment Management, Automation, Inventory Control.

SUMÁRIO

UNIVERSIDADE PAULISTA - UNIP 1
RESUMO 1
RESUMO DE LINGUA ESTRANGEIRA (INGLÊS) 3
INTRODUÇÃO 4
Objetivos 5
2.1 Objetivo geral 5
2.2 Objetivo específico 5
DESENVOLVIMENTO 6
3.1 Levantamento de Dados e Contextualização do Cliente 6
3.1.1 Gestão de Versionamento e Acompanhamento Ágil (Git e GitHub) 6
3.2 Engenharia de Requisitos 7
Requisitos Funcionais (RF) 7
Requisitos Não Funcionais (RNF) 8
3.3 Planejamento Ágil das Sprints (Product Backlog) 9
3.3.1 Gestão de Versionamento e Acompanhamento das Sprints (Git e GitHub) 10
3.4 Modelagem de Processos com UML 10
3.5 Modelagem de Banco de Dados (brModelo) e Persistência Binária 11
3.6 Implementação do Algoritmo em C (Code::Blocks) 12
3.7 Módulo de Agendamento e Gestão de Reservas 14
3.8 Infraestrutura de Redes, Ambiente Tecnológico e Segurança 15
3.9 Plano de Testes e Homologação dos Módulos 15
4 CONSIDERAÇÕES FINAIS 16
5 REFERÊNCIAS 16
6 APÊNDICE – Código-Fonte e Estruturas Modulares da Aplicação em C 17
6.1 Estrutura de Cabeçalho dos Agendamentos (agendamento.h) 17
6.2 Implementação das Funções do Módulo de Agendamento (Agendamento.c) 17

INTRODUÇÃO

Com o avanço constante da tecnologia da informação, a automatização de processos tornou-se um fator crucial para a eficiência operacional e para a competitividade de empresas de diversos setores. No segmento de prestação de serviços, como o de barbearias e estéticas masculinas, a transição de métodos tradicionais e manuais para sistemas de gestão informatizados possibilita um controle mais preciso de operações diárias, agendamentos e cadastros de clientes.

Atualmente, a Barbearia Santos enfrenta desafios operacionais decorrentes do gerenciamento manual de suas rotinas. A ausência de um sistema centralizado para o registro de clientes, controle de serviços e organização de horários pode ocasionar retrabalho, inconsistência em dados cadastrais e perda de eficiência no atendimento ao público.

Diante desse cenário, este projeto tem como objetivo propor e desenvolver um sistema de gestão sob medida para a Barbearia Santos. A implementação do software visa otimizar os processos administrativos e operacionais, garantindo maior agilidade no cadastro de usuários, confiabilidade no armazenamento das informações e facilidade no acompanhamento dos serviços prestados.

Para contemplar os aspectos técnicos e práticos do desenvolvimento, este trabalho está estruturado em seções que abordam o levantamento de requisitos funcionais e não funcionais, a modelagem de processos através de diagramas UML, a estruturação do banco de dados relacional, a definição da arquitetura de redes e a implementação da lógica do sistema na linguagem de programação C.

Objetivos

2.1 Objetivo geral

Desenvolver e documentar um sistema de gestão informatizado para a Barbearia Santos, automatizando os processos operacionais de cadastro de usuários e controle de agendamentos por meio de uma aplicação estruturada na linguagem C, integrada a uma arquitetura de banco de dados e modelo de redes adequado.

2.2 Objetivo específico

- Mapear os processos operacionais e realizar o levantamento de requisitos funcionais e não funcionais do estabelecimento.
- Modelar o fluxo do sistema e a estrutura de dados utilizando diagramas UML e o Modelo Entidade-Relacionamento (MER).
- Projetar a infraestrutura de redes necessária para garantir a segurança, conectividade e disponibilidade do sistema.
- Implementar a lógica de cadastro, validação e controle de acesso na linguagem de programação C.
- Aplicar boas práticas de Engenharia de Software Ágil para o gerenciamento do projeto e versionamento do código.

DESENVOLVIMENTO

3.1 Levantamento de Dados e Contextualização do Cliente

A Barbearia Santos é um estabelecimento comercial do ramo de estética e cuidados masculinos que busca modernizar sua gestão operacional. Atualmente, o controle dos atendimentos, o cadastro dos clientes e a agenda diária de serviços são realizados de forma manual ou com auxílio de ferramentas descentralizadas, o que gera inconsistências no fluxo de informações e limita o acompanhamento do crescimento do negócio.

Durante a etapa de levantamento de dados e análise do cenário atual, identificou-se a necessidade prioritária de implementar uma solução software centralizada. As principais dores observadas no ambiente operacional incluem:

- **Falta de padronização no cadastro:** Ausência de um histórico confiável de clientes e administradores do sistema.
- **Risco de perda de dados:** Registros em meio físico (cadernos/fichas) vulneráveis a extravios e erros de digitação.
- **Inexistência de controle de acesso:** Falta de níveis de permissão (autenticação) para separar o acesso administrativo das funções básicas do sistema.

A partir do mapeamento dessas necessidades, definiu-se o escopo inicial do software, priorizando na primeira etapa (Sprint 1) a implementação dos módulos de autenticação, cadastro e controle de acesso de usuários.

3.1.1 Gestão de Versionamento e Acompanhamento Ágil (Git e GitHub)

Em atendimento às boas práticas de gerenciamento de projetos de software, o desenvolvimento da aplicação foi acompanhado por meio de controle de versão com Git e hospedagem de código no GitHub. A metodologia ágil (Scrum) foi integrada à plataforma utilizando quadros de tarefas (GitHub Projects), permitindo o rastreamento do ciclo de vida de cada User Story e a vinculação direta dos commits aos requisitos implementados nas Sprints.

3.2 Engenharia de Requisitos

A Engenharia de Requisitos fundamenta a arquitetura e o desenvolvimento do sistema da Barbearia Santos, estabelecendo as diretrizes funcionais e as restrições técnicas necessárias para a automação dos processos operacionais. Com a evolução do projeto e a consolidação dos módulos de autenticação e agendamento, o escopo de requisitos foi expandido para contemplar a integridade das operações e a autonomia do cliente na plataforma.

Os Requisitos Funcionais (RF) descrevem o comportamento computacional e os serviços oferecidos pelo software aos seus diferentes perfis de usuários. Por sua vez, os Requisitos Não Funcionais (RNF) determinam os critérios de qualidade, desempenho, usabilidade, segurança e persistência de dados.

Requisitos Funcionais (RF)

- **RF-01 (Gestão de Usuários):** O sistema deve permitir o cadastro de clientes e administradores, coletando nome, e-mail, telefone, senha e tipo de perfil.
- **RF-02 (Autenticação e Acesso):** O sistema deve autenticar os usuários cadastrados mediante validação estrita de e-mail e senha, direcionando-os ao menu correspondente ao seu perfil.
- **RF-03 (Validação de Cadastros):** O sistema deve validar obrigatoriamente a sintaxe do e-mail (presença de '@' e '.'), a máscara do telefone (10 a 11 dígitos) e a complexidade da senha (mínimo de 6 caracteres, letra maiúscula e caractere especial).
- **RF-04 (Módulo de Agendamento):** O sistema deve disponibilizar ao cliente autenticado a escolha de agendar um horário para si próprio ou para um terceiro (beneficiário), registrando o nome do responsável.
- **RF-05 (Validação Cronológica):** O sistema deve validar as datas e horários de agendamento, impedindo marcações retroativas e realizando a checagem automática de anos bissextos e consistência de dias e meses.
- **RF-06 (Grade de Atendimento):** O sistema deve verificar a disponibilidade da agenda comercial (das 9h às 18h) em tempo real, sinalizando os status de horário: LIVRE, OCUPADO ou INDISPONÍVEL.
- **RF-07 (Consulta de Reservas):** O sistema deve permitir ao cliente consultar exclusivamente os seus agendamentos ativos armazenados no sistema (`meusAgendamentos`).

- **RF-08 (Cancelamento com Regra de Negócio):** O sistema deve permitir o cancelamento de agendamentos, exigindo antecedência mínima de 2 horas em relação ao horário marcado e gerenciando o descarte seguro do registro

Requisitos Não Funcionais (RNF)

- **RNF-01 (Persistência e Dados):** O sistema deve garantir a persistência de dados em disco por meio de arquivos binários (`cliente.dat` e `agendamentos.dat`), preservando as informações após o encerramento do programa.
- **RNF-02 (Segurança Operacional):** O sistema deve utilizar manipulação de arquivos temporários e rotinas de backup durante as operações de cancelamento para evitar corrupção ou perda de dados em disco.
- **RNF-03 (Desempenho e Resposta):** A leitura, busca e gravação de dados nos arquivos binários devem ser executadas em tempo inferior a 1 segundo, assegurando fluidez à interface do terminal.
- **RNF-04 (Usabilidade e Interface):** A interface em linha de comando (CLI) deve apresentar menus numéricos estruturados, mensagens claras de erro e resumos legíveis para confirmação de operações.
- **RNF-05 (Arquitetura Modular):** O código-fonte deve ser organizado de forma modular na linguagem C, separando definições de estruturas em arquivos de cabeçalho (`.h`) e a lógica de negócios em arquivos de implementação (`.c`).

3.3 Planejamento Ágil das Sprints (Product Backlog)

Para garantir um fluxo contínuo de entregas e alinhamento com os Requisitos Funcionais, o desenvolvimento da aplicação foi dividido em sete Sprints encadeadas, refletindo o progresso real da equipe:

- **Sprint 1 (Elicitação e Regras de Negócio) – Concluída:** Mapeamento das necessidades do cliente, definição de User Stories, requisitos funcionais e não funcionais, e criação do Product Backlog inicial.
- **Sprint 2 (Modelagem Técnica) – Concluída:** Elaboração dos Diagramas de Casos de Uso, Diagramas de Classes Estendidos, Modelo Entidade-Relacionamento (MER) e Modelo Lógico.
- **Sprint 3 (Módulo de Cadastro e Autenticação Persistente) – Concluída:** Desenvolvimento das funções de cadastro de clientes, validações avançadas (formato de e-mail, senha forte e máscara de telefone), controle de credenciais e persistência em arquivo binário (`cliente.dat`).
- **Sprint 4 (Gestão de Serviços e Agenda do Estabelecimento) – Planejada:** Implementação do cadastro de serviços oferecidos com respectiva precificação e criação da agenda administrativa do proprietário.
- **Sprint 5 (Módulo de Agendamento do Cliente) – Concluída:** Desenvolvimento da interface interativa para consulta de datas e horários disponíveis (das 9h às 18h), validação de anos bissextos e datas passadas, opção de agendar para o próprio usuário logado (resgatando o nome do arquivo `cliente.dat`) ou para terceiros, salvamento persistente em `agendamentos.dat`, além das funções de consulta (`meusAgendamentos`) e cancelamento seguro com verificação de antecedência mínima de 2 horas e rotina de backup em disco.
- **Sprint 6 (Histórico de Atendimentos) – Planejada:** Módulo de consulta e relatório de serviços realizados, integrando dados do cliente, barbeiro, data e valor.
- **Sprint 7 (Controle de Acesso, Segurança e Testes Globais) – Planejada:** Implementação de permissões rigorosas entre perfis (Cliente vs. Dono), rotinas de backup de dados e execução da bateria final de testes de segurança e usabilidade.

3.3.1 Gestão de Versionamento e Acompanhamento das Sprints (Git e GitHub)

Para garantir o alinhamento e a colaboração contínua da equipe, a gestão do projeto foi centralizada na plataforma GitHub. A metodologia ágil (Scrum) foi operacionalizada por meio da ferramenta **GitHub Projects**, permitindo o rastreamento do progresso de cada

Sprint (To Do, In Progress, Done) e a vinculação direta de commits às *User Stories* registradas nas *Issues*.

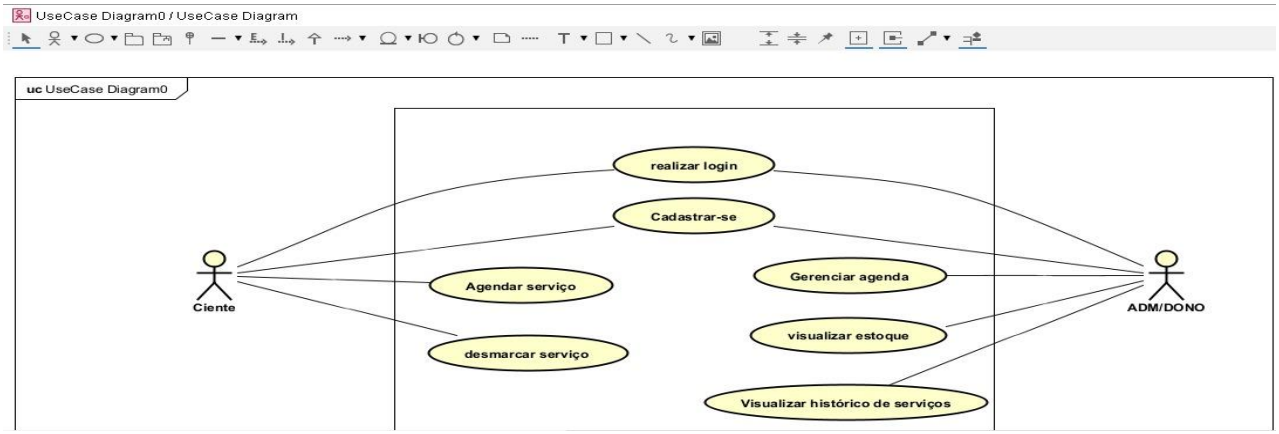
O controle de versão do código-fonte foi mantido utilizando o fluxo de trabalho Git, garantindo o histórico de alterações, o rastreamento de entregas por desenvolvedor e o armazenamento seguro da aplicação modular em C e seus respectivos arquivos de persistência binária.

Repositório do Projeto: https://github.com/SistemasSw7/PIM_Documentos/blob/main/documenta%C3%A7%C3%A3o/README.md

3.4 Modelagem de Processos com UML

A modelagem orientada a objetos foi estruturada por meio da linguagem UML (Unified Modeling Language) com o auxílio da ferramenta Astah UML. O Diagrama de Casos de Uso da Sprint 1 (Figura 1) mapeia as interações entre os atores do sistema (Cliente e Proprietário/Administrador) e as rotinas de autenticação e gestão de perfil:

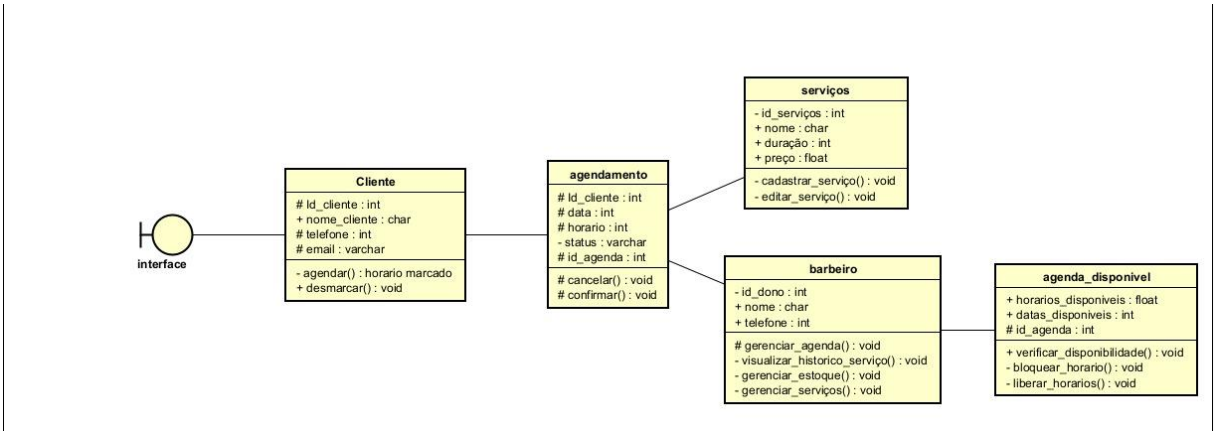
Figura 1 – Diagrama de Casos de Uso da Sprint 1



Complementarmente, o Diagrama de Classes (Figura 2) define a estrutura lógica dos

objetos, contendo a classe Usuário com seus respectivos atributos privativos e métodos públicos de manipulação e validação.

Figura 2 – Diagrama de Classes da Sprint 1:

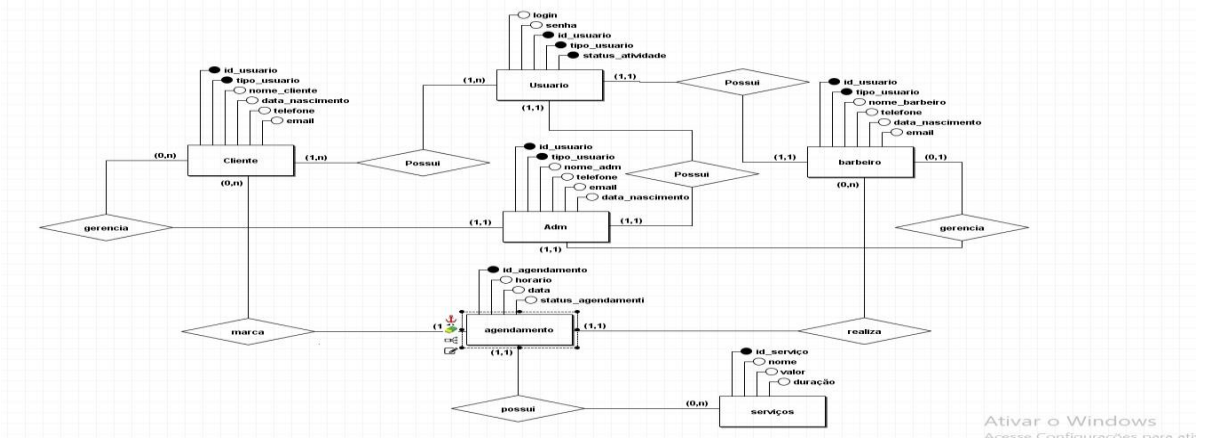


3.5 Modelagem de Banco de Dados (brModelo) e Persistência Binária

Diferente de abordagens rudimentares que armazenam registros apenas na memória volátil (RAM), o sistema da Barbearia Santos evoluiu para uma arquitetura de armazenamento binário em disco. Utilizando ponteiros de arquivo da biblioteca padrão (`FILE*`) e as funções `fwrite` e `fread`, a aplicação grava e recupera registros estruturados diretamente nos arquivos binários **`cliente.dat`** e **`agendamentos.dat`**. Essa abordagem garante que as informações cadastrais e as reservas sejam preservadas mesmo após o encerramento da aplicação, atendendo diretamente aos Requisitos Não Funcionais de confiabilidade e integridade de dados.

Essa abordagem garante que as informações cadastrais e os agendamentos sejam preservados mesmo após o encerramento da aplicação, atendendo diretamente ao Requisito Não Funcional de confiabilidade e integridade de dados.

Figura 3 – Modelo Entidade-Relacionamento (MER) para Autenticação



3.6 Implementação do Algoritmo em C (Code::Blocks)

A lógica computacional desenvolvida reflete a conclusão das **Sprints 1, 2, 3 e 5** do projeto. O código foi totalmente refatorado para uma estrutura modularizada e organizada em diretórios de cabeçalhos (`.h`) e arquivos de implementação (`.c`), englobando as rotinas de cadastro, autenticação, grade de horários e gestão de agendamentos.

1. Arquivos de Cabeçalho (Cliente.h e Agendamento.h): Definem as estruturas de dados (struct Cliente e struct Agendamento) com id, nome, e-mail, telefone, senha e campos de data/horário.

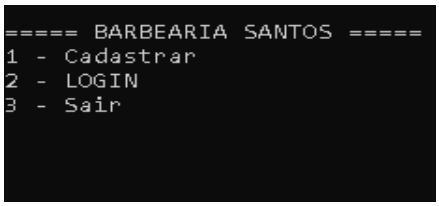
2. Módulo de Cadastro (Cadastro.c): Executa a geração automática de ID incremental, a verificação de e-mail duplicado em arquivo, a obrigatoriedade dos caracteres '@' e '.', a validação de telefone de 10 a 11 dígitos numéricos e a imposição de senhas fortes (mínimo de 6 caracteres, pelo menos uma letra maiúscula e um caractere especial).

3. Módulo de Autenticação (Login.c): Realiza a leitura sequencial do arquivo binário cliente.dat, autenticando e-mail e senha cadastrados.

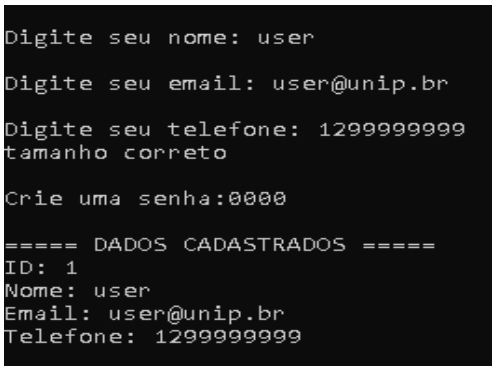
4. Módulo de Navegação (main.c e MenuCliente.c): Gerencia o fluxo de telas no terminal (CLI) e direciona o usuário autenticado para as opções de agendamento e gestão de conta. Abaixo, apresentam-se as telas do sistema compondo a execução real no terminal:

Abaixo, apresentam-se as telas do sistema compondo a execução real no terminal:

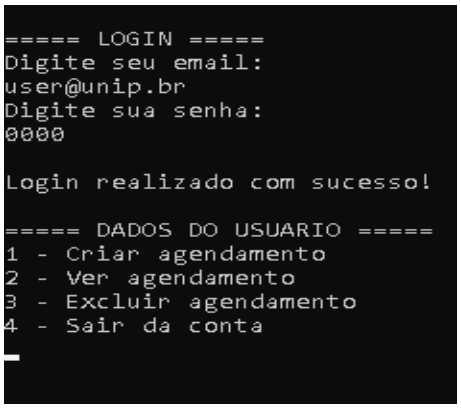
Menu Principal e Seleção de Funções



Cadastro de Cliente com Validação Rigorosa de E-mail, Telefone e Senha Forte



Tela de Autenticação e Menu Exclusivo da Área do Cliente



3.7 Módulo de Agendamento e Gestão de Reservas

O sistema evoluiu para incorporar o módulo completo de agendamentos e atendimento autônomo pelo cliente. A estrutura de dados foi dimensionada por meio da struct Agendamento, contemplando identificador único, vínculo com o cliente, nome do beneficiário do atendimento, data detalhada (dia, mês e ano) e horário da reserva.

As principais rotinas desenvolvidas neste módulo englobam:

• Seleção de Perfil de Atendimento: O cliente autenticado pode optar por agendar um horário para si mesmo (resgatando automaticamente seu nome cadastrado no arquivo cliente.dat) ou para terceiros, informando o nome e sobrenome do beneficiário.

• Validação Cronológica Avançada: O algoritmo valida a consistência de dias, meses e anos, realizando o cálculo automático de anos bissextos e bloqueando rigorosamente datas ou horários retroativos com base na data atual obtida pelo sistema operacional (time.h).

• Grade de Horários Dinâmica: O sistema lê o arquivo binário agendamentos.dat, cruza os dados com o horário comercial da barbearia (das 9h às 18h) e exibe em tempo real o status de cada horário: LIVRE, OCUPADO ou INDISPONÍVEL (caso o horário do dia atual já tenha expirado).

• Consulta e Cancelamento com Regra de Negócio: Os clientes possuem acesso à listagem de suas reservas ativas (meusAgendamentos) e à rotina de exclusão (cancelarAgendamento). Para garantir a segurança operacional, o cancelamento utiliza manipulação de arquivos temporários e rotinas de backup em disco, exigindo obrigatoriamente que a solicitação seja feita com antecedência mínima de 2 horas em relação ao horário agendado (calculado via difftime).

3.8 Infraestrutura de Redes, Ambiente Tecnológico e Segurança

Para garantir a conectividade, a segurança e a disponibilidade operacional do sistema da Barbearia Santos, foi dimensionada uma infraestrutura tecnológica adequada aos requisitos da aplicação:

• Arquitetura de Rede e Conectividade: O sistema adota uma arquitetura em ambiente local (standalone), sendo executado diretamente na estação de trabalho ou terminal. No entanto, a infraestrutura de redes foi projetada contemplando futura expansão para um modelo cliente-servidor em rede local (LAN) ou ambiente em nuvem, permitindo o acesso remoto seguro por meio de protocolos TCP/IP e isolamento de tráfego entre os clientes e o servidor de dados da barbearia.

• Ambiente de Desenvolvimento e Execução: A aplicação foi desenvolvida utilizando a IDE Code::Blocks com o compilador GCC (padrão C11), garantindo alta portabilidade, desempenho otimizado e compatibilidade com sistemas operacionais Windows e Linux.

• Segurança e Integridade de Acesso: A segurança do ambiente é respaldada pela separação estrita de privilégios entre perfis (Cliente vs. Administrador) e pelo armazenamento persistente em disco por meio de arquivos binários (cliente.dat e agendamentos.dat), o que previne acessos não autorizados diretos e assegura a consistência transacional das informações.

3.9 Plano de Testes e Homologação dos Módulos

Como etapa fundamental da Engenharia de Software, o sistema foi submetido a uma rigorosa bateria de testes funcionais e estruturais para validar a conformidade com os requisitos especificados:

• Testes de Cadastro e Validação de Credenciais: Foram injetados dados intencionalmente incorretos (e-mails sem os caracteres obrigatórios '@' e '.', números de telefone com contagem incorreta de dígitos e senhas abaixo do comprimento mínimo ou sem caracteres especiais) para verificar a eficácia das restrições implementadas em Cadastro.c.

• Testes de Consistência Cronológica e Agendamento: O módulo de agendamentos foi validado frente a cenários de anos bissextos, datas retroativas e horários fora do expediente comercial estabelecido (das 9h às 18h), confirmando o correto bloqueio de marcações inválidas.

• Homologação da Regra de Cancelamento: A regra de antecedência mínima de 2 horas foi rigorosamente testada com simulações temporais baseadas na biblioteca time.h, atestando o bloqueio de cancelamentos fora do prazo regulamentar e a correta execução da rotina de salvamento com arquivo temporário (agendamentos_temp.dat).

4 CONSIDERAÇÕES FINAIS

O desenvolvimento do Projeto Integrado Multidisciplinar (PIM II) permitiu a aplicação prática e integrada dos conceitos abordados nas disciplinas do semestre no curso de Análise e Desenvolvimento de Sistemas. O projeto solucionou os gargalos operacionais da Barbearia Santos ao substituir controles informais por uma aplicação segura, padronizada e modular em linguagem C.

A evolução do projeto demonstrou um avanço significativo em relação ao planejamento inicial. A transição da alocação temporária em memória para a persistência em arquivos binários em disco (`cliente.dat` e `agendamentos.dat`), aliada à criação de estruturas (`structs`) e à divisão modular por arquivos de cabeçalho (`.h`), elevou o nível técnico da solução entregue. As rotinas de cadastro com validação rigorosa de credenciais, o controle de acesso por perfil e o módulo completo de agendamento autônomo com validação cronológica e regra de cancelamento foram testados e homologados com sucesso. A divisão em sete Sprints pela metodologia ágil permitiu a entrega avançada do

núcleo e das reservas do sistema nas **Sprints 1, 2, 3 e 5**, criando a infraestrutura necessária para a continuidade das etapas finais.

A divisão em sete Sprints pela metodologia ágil permitiu a entrega completa do núcleo do sistema nas Sprints 1 a 3, criando a infraestrutura necessária para a continuidade das etapas de agendamento e relatórios operacionais. Conclui-se que os objetivos da entrega foram plenamente alcançados, reforçando a importância da engenharia de software e da tecnologia na automação de pequenos negócios.

5 REFERÊNCIAS

BEZERRA, Eduardo. Princípios de Análise e Projeto de Sistemas com UML. 3. ed. Rio de Janeiro: Elsevier, 2015.

DEITEL, Paul; DEITEL, Harvey. C: Como Programar. 6. ed. São Paulo: Pearson Education do Brasil, 2011.

HEUSER, Carlos Alberto. Projeto de Banco de Dados. 6. ed. Porto Alegre: Bookman, 2009.

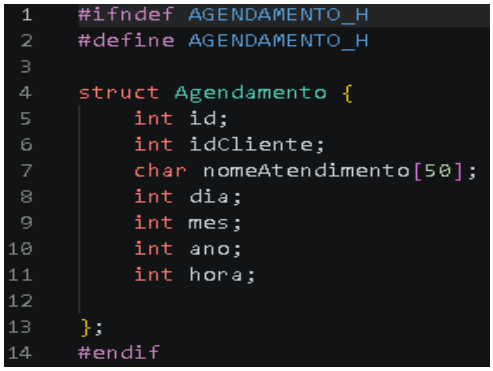
PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.

SANTOS, Rafael. brModelo: Ferramenta Acadêmica para Modelagem de Banco de Dados Relacional. Disponível em: <http://www.sis4.com/brmodelo/>. Acesso em: 04 out. 2026

6 APÊNDICE – Código-Fonte e Estruturas Modulares da Aplicação em C

O sistema da Barbearia Santos foi estruturado de forma modular em linguagem C, separado em arquivos de cabeçalho (.h) para definição de estruturas e arquivos de implementação (.c) para a lógica de negócio e persistência binária em disco.

6.1 Estrutura de Cabeçalho dos Agendamentos (agendamento.h)



```h
#ifndef AGENDAMENTO_H
#define AGENDAMENTO_H

struct Agendamento {
    int id;
    int idCliente;
    char nomeAtendimento[50];
    int dia;
    int mes;
    int ano;
    int hora;

};
#endif
```

6.2 Implementação das Funções do Módulo de Agendamento (Agendamento.c)

O arquivo de implementação Agendamento.c centraliza a lógica operacional do sistema de reservas, estruturando-se em rotinas essenciais que asseguram a autonomia do cliente e a integridade dos dados em disco:

A função agendarHorario gerencia o fluxo completo de novas marcações. Inicialmente, o sistema captura a data e o horário correntes do sistema operacional por meio da biblioteca padrão time.h. O usuário pode optar por registrar o atendimento para si próprio — recuperando automaticamente o nome cadastrado no arquivo de clientes — ou indicar um beneficiário terceiro. Em seguida, o algoritmo executa validações cronológicas rigorosas, incluindo a verificação matemática de anos bissextos e o bloqueio automático de datas ou horários retroativos. Através da varredura no arquivo binário agendamentos.dat, o programa cruza as informações com o expediente

comercial (das 9h às 18h) para evitar conflitos de horários, realizando a gravação persistente dos dados de maneira segura.

Para o acompanhamento das reservas ativas, a função meusAgendamentos realiza a leitura e o filtro no arquivo binário, exibindo de forma limpa e organizada no terminal exclusivamente os agendamentos vinculados ao identificador do cliente autenticado.

Por fim, a função cancelarAgendamento assegura o gerenciamento de exclusões sob critérios rigorosos de segurança. Ao requisitar o cancelamento, o sistema calcula a diferença exata de tempo até o horário agendado utilizando a função difftime. Caso a solicitação ocorra com antecedência inferior ao limite estabelecido de 2 horas, a operação é negada. Estando dentro do prazo regulamentar, o registro é removido com o

auxílio de um arquivo temporário (agendamentos_temp.dat) e rotinas de backup, blindando a aplicação contra perdas ou corrupções de dados.